MANIPAL UNIVERSITY

School of Computer Science and Engineering
Department of Computer Science and Engineering

# *EnFeSTDroid*: Ensembled feature selection techniques based Android malware detection

Submitted by :
Dhwani Jain
2427030681

Supervised by :
Dr. Neha Janu

# Outline

INTRODUCTION

LITERATURE REVIEW

PROBLEM STATEMENT

PROPOSED SOLUTIONS

OBJECTIVES

PROPOSED METHODOLOGY

OUTCOMES

REFERENCES

# Introduction

Android is the most widely used mobile operating system, making it a major target for malware attacks such as ransomware, spyware, and malicious applications. Traditional detection techniques often rely on limited or single feature selection methods, which may fail to capture all important characteristics of malware. As applications become more complex, identifying the most relevant features becomes essential for accurate detection. Static analysis using permissions is efficient but may include redundant or irrelevant features, reducing model performance.

Solution - An ensemble feature selection approach that combines multiple feature selection techniques with machine learning models to improve accuracy and reliability in Android malware detection.

| Analysis Types | What it checks | Strengths | Weakness |
|---|---|---|---|
| Static Analysis | Permissions, API calls (without running the app) | Fast, lightweight | Can be fooled by obfuscation |
| Dynamic Analysis | System calls, runtime behaviour, network activity (while app runs) | Detects actual malicious behaviour | Slower, resource-heavy |

**Why Ensemble Feature Selection?**

- Combines multiple feature selection techniques
- Captures important features missed by individual methods
- Reduces bias and improves reliability

# Literature Review

**Title -** *EnFeSTDroid* : Ensembled feature selection techniques based Android malware detection

**Author -** Suruchi Jain, Hemant Goyal, Anshul Arora, Dhirendra Kumar

**Year :** January 2026

**Findings :**

- Combining multiple feature selection techniques (6 methods) gives better results than relying on a single method.
- Permissions are highly effective features, and ensemble selection improves identification of important ones, boosting accuracy.
- The proposed model achieved 96.27% accuracy, outperforming many existing approaches.
- Feature selection not only improves accuracy but also reduces overfitting, computational cost, and enhances interpretability.

# Literature Review

**Title :** Android Malware Detection Using API Calls and Permissions With Random Forest Classifier

**Author :** Abdul Museeb, Yaman Hamed, Yahia Baashar, Abdulkarim M. Jamal Kanaan-Jebna, Anis Amazigh Hamza and Rajalingam Sokkalingam

**Year :** Received 11 November 2025, accepted 29 December 2025, date of publication 12 January 2026, date of current version 15 January 2026.

**Findings :**

- A Random Forest–based static model using API calls and permissions achieved 95% detection accuracy, proving high effectiveness.
- Feature selection (correlation + PCA) significantly reduces dimensionality while maintaining performance, improving efficiency.
- The model is fast and lightweight (training <2s, inference <0.1s), making it suitable for real-world and on-device deployment.
- Random Forest outperformed other models (SVM, Logistic Regression, XGBoost) and provided better interpretability via feature importance.

# Literature Review

**Title :** Feature-driven static analysis for learning-based android malware detection: A review

**Author :** Sumesh Kharnotia, Bhavna Arora, Ravdeep Kour

**Year :** February 2026

**Findings :**

- Static analysis (without executing apps) is widely used and effective, but current research lacks depth in feature engineering, visualization, and reverse engineering techniques.
- Machine Learning and Deep Learning models dominate, but no single approach is universally optimal—performance depends heavily on feature selection and datasets.
- There are major research gaps, especially in lightweight, efficient, and real-time detection systems.
- The paper emphasizes the need for better feature extraction, hybrid methodologies, and standardized evaluation frameworks for future improvements.

# Problem Statement

Existing Android malware detection systems often rely on single feature selection techniques, which may fail to identify all relevant features and reduce detection accuracy. Therefore, there is a need to develop a system that effectively combines multiple feature selection methods to improve classification performance and accurately detect malware applications.

# Objectives

- To implement an Android malware detection system
- To apply multiple feature selection techniques
- To identify the most relevant permission features
- To improve classification accuracy using ML models
- To evaluate performance using standard metrics

# Proposed Methodology

**Step 1 – Data Collection**

**Dataset Used -** CIC-AndMal2017 (Upgraded Version)

**Source -** Mendeley Data Repository

**Total samples** – 11,598 Android applications

**Features** – 470 features (static + dynamic)

**Classes** – 5 categories

**Class 0** – Benign (safe applications)

**Class 1–4** – Malware families (Adware, Banking malware, SMS malware, Riskware)

**Why this dataset?**

- Updated and enhanced version of CIC malware dataset (2026 version)
- Contains both static and dynamic features for better analysis
- Ready-to-use CSV format – no manual feature extraction required
- Well-labeled and suitable for machine learning models

# Proposed Methodology

**Step 2 – Data Preparation**

**Tools Used :**

- Visual Studio Code (Python)
- Libraries: pandas, numpy, scikit-learn, scipy
- **Steps Performed :**

1. Imported dataset (CSV file) into the working environment
2. Loaded dataset using pandas
3. Identified label column – 'Class' (Benign/Malware)
4. Separated features (X) and labels (y)
5. Verified and retained numeric features (470 features)
6. Handled missing values – filled with 0
7. Split data :
    - 80% training
    - 20% testing
    - Stratified split maintained class distribution

# Proposed Methodology

**Step 3 – Feature Selection**

**Why Feature Selection?**

- Reduces irrelevant and redundant features
- Improves model accuracy and efficiency
- Handles high-dimensional data effectively

**Techniques Used :**

1. Information Gain
2. Chi-Square
3. Extra Tree Classifier
4. Mean Term Frequency (MTF)
5. Inverse Document Frequency (IDF)
6. MTF-IDF

**Approach :**

- Applied all feature selection techniques on dataset
- Ranked features based on importance scores
- Combined rankings from all techniques
- Selected optimal subset of important features

# Proposed Methodology

**Step 4 – Rank Fusion**

**Why Rank Fusion?**

- Different feature selection techniques produce different rankings
- Important features may vary across methods
- Single method can introduce bias

**Approach :**

1. Applied multiple feature selection techniques
2. Generated ranked feature lists from each method
3. Created a rank matrix combining all rankings
4. Applied Friedman Test to check significance
5. Aggregated ranks to obtain final feature list

**Friedman Test :**

- A statistical test used to compare multiple methods
- Checks whether rankings are significantly different
- If significant, then combine rankings for better results

# Proposed Methodology

**Step 5 – Model Training**

**Why Random Forest?**

- Handles high-dimensional data well (470 features)
- Resistant to overfitting due to ensemble learning
- Works well with selected features after feature selection
- Fast to train and easy to interpret (Showed Highest Acurracy)

**Code :**

```
model = RandomForestClassifier(n_estimators=100,max_depth=15,random_state=42,n_jobs=-1)
model.fit(X_train_selected, y_train)
```

- n_estimators = 100 (100 decision trees)
- Training time: ~10-20 seconds

# Proposed Methodology

**Step 6 – Model Evaluation**

**Test Set :** 20% unseen apps

**Metrics Calculated :**

- **Accuracy** = (Correct predictions) / (Total predictions)
- **Precision** = When model says "malware", how often is it right?
- **Recall** = What percentage of actual malware did the model catch?
- **F1-score** = Harmonic mean of precision and recall

**Code for evaluation :**

```
y_pred = model.predict(X_test_selected)

acc = accuracy_score(y_test, y_pred)
report = classification_report(y_test, y_pred, output_dict=True)
```

# Results – Model Performance

**Model Performance on Test Set :**

| Metric | Value |
|---|---|
| Accuracy | 0.97 |
| Precision | 0.96 |
| Recall | 0.96 |
| F1 – Score | 0.96 |

**Interpretation :**

- 97 out of 100 apps were correctly classified
- When the model flagged an app as malware, it was correct 96% of the time
- The model caught 96% of all actual malware apps
- Excellent performance – shows model effectively detects malware

**Output :**

```
Accuracy: 0.97

Class-wise Performance :

Class 0 (Benign):
  Precision: 0.98
  Recall   : 0.98
  F1-Score : 0.98

Class 1 (Malware):
  Precision: 0.96
  Recall   : 0.96
  F1-Score : 0.96

Overall Performance:
  Macro Avg     : 0.97
  Weighted Avg  : 0.97
```

# Outcomes

**Achievements for this semester :**

| Outcome | Status |
|---|---|
| Ensemble feature selection model implemented | Completed |
| Ensemble feature selection model implemented | Completed |
| Ensemble feature selection model implemented | Completed |
| Ensemble feature selection model implemented | Completed |

**Planned for next semester :**

| Outcome | Target |
|---|---|
| Research Paper | Drafted / Under review |
| Copyright | To be filed |
| Real-time deployment | Partially completed – prototype ready |

# References


1) https://www.sciencedirect.com/science/article/pii/S0045790625007062?ref=pdf_download&fr=RR-2&rr=9e593d6a0f822b0d

2) https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=11338752

3) https://www.sciencedirect.com/science/article/pii/S2405959526000056?ref=pdf_download&fr=RR-2&rr=9eb42d284a90bc88

4) https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=10944807

5) https://dl.acm.org/doi/10.1145/3638552

6) https://ieeexplore.ieee.org/abstract/document/9130686

7) https://data.mendeley.com/datasets/bpkksc9v5s/5

# Thank You